Machine Learning Models Hub: A Multi-Model Streamlit Application for Interactive Predictive Analytics

Rachel Monique Tumulak
School of Advanced Technology
Algonquin College
Ottawa, Canada
tumu0009@algonquinlive.com

***Abstract*—This paper presents the design and implementation of *Machine Learning Models Hub*, a multi-page Streamlit application that integrates regression, classification, clustering, and neural-network models into a single interactive system. The project was developed by adapting instructor-provided Jupyter notebooks into a deployable web interface, allowing end users to perform single-record inference, batch prediction, and model analysis from a browser. The final application includes four analytical modules: real estate price prediction, loan eligibility prediction, customer segmentation, and graduate admission prediction. In addition to describing the software architecture, this paper highlights the data flow from notebook-based experimentation to trained-model serialization and front-end deployment.**

***Index Terms*—Machine learning, Streamlit, regression, classification, clustering, neural networks, deployment, interactive analytics**

I. Introduction

Machine learning models are often developed in notebooks for experimentation, yet end users rarely interact with those models directly. A common challenge in applied machine learning is converting notebook-based prototypes into usable applications that support real input, visualization, and repeatable inference. This project addresses that gap by packaging several machine learning tasks into one web application called *Machine Learning Models Hub*.

The application combines four academic case studies into a single platform: real estate price prediction, loan eligibility classification, customer segmentation, and graduate admission prediction. Instead of leaving the work at the notebook stage, the project transforms the trained models into a multi-page Streamlit application with a shared navigation layer, model-specific interfaces, and support for batch processing.

II. Project Background and Scope

The system was built from instructor-provided notebooks and then adapted into a deployable application. The uploaded notebooks cover four workflows: a loan eligibility classification notebook, a real estate regression notebook, an unsupervised clustering notebook for mall customers, and a UCLA admission notebook for neural networks. The notebooks show the training-side scope of each problem and were used as the analytical starting point for the app implementation.

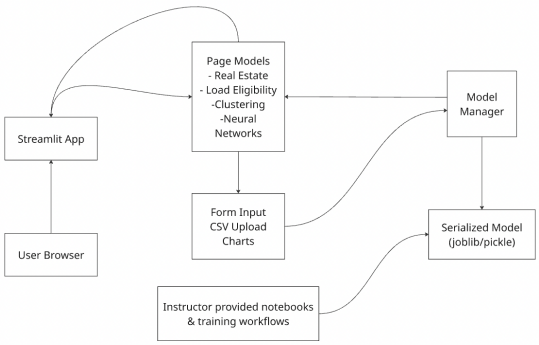

Fig. 1. Overall architecture of the Machine Learning Models Hub. Notebook workflows are used to train models, which are then serialized and consumed by the Streamlit interface for interactive inference.

The deployed application exposes five sidebar pages: a home dashboard, real estate pricing, loan eligibility, customer segmentation, and neural networks. The main Streamlit entry point routes the user to the corresponding page module based on sidebar selection. Each module then loads a trained model and provides prediction, batch upload, and analysis views where appropriate.

III. System Architecture

The complete system follows a layered architecture that separates user interaction from model inference and training artifacts.

Figure 1 summarizes the architecture. The notebook layer is responsible for model development and experimentation. Once trained, models are saved and later loaded by the application. The Streamlit app handles the presentation layer, while page modules implement task-specific logic. This organization

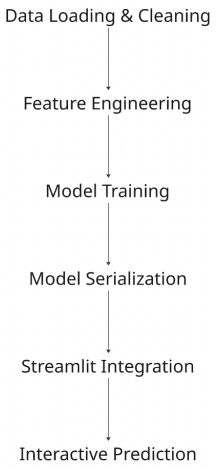

Fig. 2. Workflow used to adapt instructor-provided notebooks into the deployed Streamlit system.

makes the system easier to extend because new models can be added by training them offline and then connecting a new page to the loading utility.

A. Application Routing Layer

The main application entry file configures the page, custom styling, and sidebar navigation. The sidebar exposes four task-specific modules plus a home page, and the selected page determines which `show()` function is executed. This routing design keeps the application modular and reduces coupling between tasks.

B. Page-Level Inference Modules

Each page contains three common responsibilities:

1) collecting user input through widgets,
2) preparing feature vectors for the trained model, and
3) displaying predictions and explanatory graphics.

The real estate and loan modules additionally support model comparison between two algorithms. The clustering and neural-network modules provide task-specific analysis views, such as cluster visualization or probability-based admission feedback.

C. Notebook-to-Application Pipeline

The transition from notebook development to deployed application can be summarized as the pipeline in Fig. 2.

IV. Implemented Analytical Modules

A. Real Estate Price Prediction

The real estate module is a supervised regression task. In the Streamlit page, the user enters features such as year sold, property tax, insurance, bedrooms, bathrooms, square footage, year built, lot size, basement status, location popularity, recession indicator, and property type. The page compares a linear regression model and a random forest regressor.

The module is useful because it demonstrates both a linear baseline and a nonlinear ensemble approach for property-value estimation. The batch mode also allows a CSV file of properties to be scored at once.

B. Loan Eligibility Prediction

The loan module is a binary classification problem. It uses applicant income, co-applicant income, loan amount, loan term, credit history, gender, marital status, dependents, education, self-employment status, and property area. Two models are exposed to the user: logistic regression and random forest classification.

This module emphasizes the difference between interpretable linear classification and more flexible ensemble-based classification. The application also presents approval probability to support comparative decision analysis rather than only a hard class label.

C. Customer Segmentation

The customer-segmentation module applies K-means clustering to mall-customer data. The visible user inputs are age, annual income, and spending score. The system predicts the cluster for a single customer or assigns segments to an uploaded dataset. In the analysis tab, the application displays cluster statistics and scatter-based segment visualization.

This module shows how unsupervised learning can be integrated into the same application structure used for supervised tasks, even though the output is a segment label rather than a class or numeric prediction.

D. Graduate Admission Prediction Using Neural Networks

The neural-network module predicts graduate-admission outcomes using a multilayer perceptron classifier. According to the deployed page, the inputs include GRE score, TOEFL score, university rating, statement of purpose, letter of recommendation strength, CGPA, and research experience. The implementation converts some inputs to one-hot encoded features before prediction and reports class probabilities to the user.

This module is especially important because it extends the platform beyond conventional scikit-learn regression and classification baselines into a neural-network workflow derived from the UCLA admission notebook.

V. Model Summary

Table I summarizes the main analytical tasks implemented in the application.

VI. Implementation Details

The application is structured as a multi-page Streamlit system with a shared home page and model-specific pages. The home module presents an overview of available models and usage instructions. The task pages expose prediction workflows and charts. This design provides a consistent user

TABLE I
SUMMARY OF MACHINE LEARNING MODULES

Module	Task	Algorithms
Real Estate	Regression	Linear Regression, Random Forest Regressor
Loan Eligibility	Classification	Logistic Regression, Random Forest Classifier
Customer Segmentation	Clustering	K-means
Graduate Admission	Classification	Multilayer Perceptron

experience while still allowing each machine learning problem to have domain-specific features and outputs.

Another notable implementation detail is the support for batch processing. In the loan, real estate, clustering, and neural-network modules, the user can upload CSV data and receive predictions for multiple records. This makes the application more practical than a single-form demo because it supports small-scale analytical workflows.

VII. DISCUSSION

The project demonstrates that instructor-provided notebooks can serve as the experimental foundation of a deployed analytical system. The main contribution is not the invention of new models, but the conversion of classroom model-development artifacts into a usable interface for end users.

The resulting application also makes model comparison more tangible. For instance, the loan and real estate pages allow side-by-side use of two algorithms, helping users understand trade-offs between interpretability and nonlinear modeling. Likewise, the clustering and neural-network modules broaden the scope of the system by including unsupervised learning and deep-learning-inspired classification in the same interface.

VIII. LIMITATIONS

Several limitations remain. First, the quality of predictions depends on the quality of the original training data and preprocessing completed in the notebooks. Second, the deployed system is primarily an educational platform rather than a production-grade decision-support tool. Third, dependency compatibility can affect deployment when machine learning libraries and Python versions are not aligned.

A further technical limitation is that some preprocessing logic appears to be reproduced inside the app pages rather than loaded from a fully serialized end-to-end pipeline. In a production setting, preprocessing objects such as encoders and scalers should be saved and reused directly during inference to avoid train–serve mismatch.

IX. CONCLUSION

This paper presented the design of *Machine Learning Models Hub*, a unified Streamlit application that deploys four machine learning workflows adapted from instructor-provided notebooks. By combining regression, classification, clustering, and neural-network prediction into one interface, the project demonstrates a complete educational pipeline from experimentation to interactive deployment. The architecture is modular, extensible, and well suited for coursework that emphasizes practical machine learning implementation.

Future improvements may include automated retraining, stronger evaluation reporting, explainability methods, persistent model metadata, and deployment hardening through fully serialized preprocessing pipelines.

REFERENCES

[1] F. Pedregosa *et al.*, "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[2] T. Kluyver *et al.*, "Jupyter Notebooks—a publishing format for reproducible computational workflows," in *Positioning and Power in Academic Publishing*, 2016, pp. 87–90.

[3] W. McKinney, "Data Structures for Statistical Computing in Python," in *Proc. 9th Python in Science Conf.*, 2010, pp. 56–61.

[4] J. D. Hunter, "Matplotlib: A 2D Graphics Environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007.

[5] Streamlit, "Streamlit documentation." [Online]. Available: https://streamlit.io. Accessed: Mar. 20, 2026.